

LAB2 - Programação para Sistemas Paralelos e Distribuídos T02

Estudo sobre Hadoop e Spark

ANA LUIZA RODRIGUES DA SILVA - 211030676
GUILHERME BRITO VILAS BOAS - 190108011

LAÍS RAMOS BARBOSA - 170107574
LUCAS LOPES ROCHA - 202023903
MATHEUS RAPHAEL SOARES DE OLIVEIRA - 190058587

Contents

1	Introdução	2
2	Conhecendo o Apache Hadoop	2
2.1	Montagem de um cluster Hadoop básico	2
2.1.1	Arquitetura do cluster	2
2.1.2	Containers	2
2.2	Estrutura de arquivos	2
2.3	Configurações principais	3
2.3.1	HDFS (Hadoop Distributed File System)	3
2.3.2	YARN (Resource Manager)	3
2.3.3	MapReduce	3
2.4	Interfaces Web	3
2.4.1	1. Namenode Web UI	3
2.4.2	2. ResourceManager Web UI	4
2.5	Execução	4
2.5.1	Executando Word Count (Python)	4
2.5.2	Executando Word Count (Java)	7
2.6	Materiais utilizados de base	7
2.6.1	Configurações Implementadas	8
2.7	Teste de Tolerância a faltas e performance de aplicações Hadoop	17
2.7.1	Teste 1: Baseline	17
2.7.2	Teste 2: Falha pré-existente	21
2.7.3	Teste 3: Falha em tempo real	29
2.7.4	Conclusão	39

3	Conhecendo o Apache Spark	39
4	Conclusão	39

1 Introdução

2 Conhecendo o Apache Hadoop

2.1 Montagem de um cluster Hadoop básico

Este repositório contém a implementação completa de um cluster Hadoop básico usando containers Docker, incluindo:

- 1 nó master (Namenode)
- 2 nós slaves (Datanodes)
- HDFS configurado e testado
- Interfaces Web funcionais (Namenode, ResourceManager)
- Exemplo WordCount

2.1.1 Arquitetura do cluster

```

1
2
3          MASTER NODE (Namenode)
4  - Gerencia metadados do HDFS
5  - Web UI: http://localhost:9870
6  - ResourceManager (YARN): http://localhost :8088
7
8
9
10
11
12      SLAVE NODE 1      SLAVE NODE 2
13      (Datanode1)      (Datanode2)
14      - DataNode        - DataNode
15      - NodeMgr         - NodeMgr

```

2.1.2 Containers

namenode: Master node + ResourceManager **datanode1 e datanode2:** Workers para HDFS **nodemanager1 e nodemanager2:** Workers para YARN **resourcemanager:** Gerenciador de recursos **historyserver:** Histórico de jobs

2.2 Estrutura de arquivos

```

1 hadoop-cluster/
2 docker-compose.yml           # Definição dos serviços
3 hadoop.env                   # Configurações Hadoop
4 scripts/
5     mapper.py                # Mapper para WordCount
6     reducer.py               # Reducer para WordCount
7     WordCount.java           # Versão Java
8 dados/
9     teste.txt                 # Arquivo de teste

```

2.3 Configurações principais

2.3.1 HDFS (Hadoop Distributed File System)

Parâmetro	Valor	Descrição
fs.defaultFS	hdfs://namenode:9000	URI do filesystem
dfs.replication	2	Fator de replicação
dfs.namenode.name.dir	/hadoop/dfs/name	Dir do Namenode
dfs.datanode.data.dir	/hadoop/dfs/data	Dir dos Datanodes

2.3.2 YARN (Resource Manager)

Parâmetro	Valor	Descrição
yarn.resourcemanager.hostname	resourcemanager	Hostname do RM
yarn.nodemanager.resource.memory-mb	4096	RAM por NodeManager
yarn.nodemanager.resource.cpu-vcores	4	CPUs por NodeManager

2.3.3 MapReduce

Parâmetro	Valor	Descrição
mapreduce.framework.name	yarn	Framework usado
mapreduce.map.memory.mb	512	RAM para Map tasks
mapreduce.reduce.memory.mb	512	RAM para Reduce tasks

2.4 Interfaces Web

2.4.1 1. Namenode Web UI

- URL: <http://localhost:9870>

- **Funcionalidades:**
 - Status do HDFS
 - DataNodes ativos
 - Uso de espaço
 - Explorador de arquivos
 - Logs do sistema

Como acessar:

1. Abra o navegador
2. Acesse `http://localhost:9870`

2.4.2 2. ResourceManager Web UI

- **URL:** `http://localhost:8088`
- **Funcionalidades:**
 - Aplicações em execução
 - Filas de recursos
 - Jobs MapReduce
 - Uso de CPU/Memória
 - Histórico

Como monitorar um job:

1. Acesse `http://localhost:8088`
2. Clique em “Applications”
3. Clique no ApplicationID do job em execução
4. Veja progresso de Map e Reduce

2.5 Execução

```
1 cd hadoop-cluster
2 docker compose up -d
3
4 #verificar estado dos containers
5 docker ps
```

2.5.1 Executando Word Count (Python)

1. Instalar o python nos containers `nodemanager1`, `nodemanager2`, `datanode1`, `datanode2`

```
1 docker exec -it nodemanager1 bash -c "
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/
  sources.list && \
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates
  main' >> /etc/apt/sources.list && \
4 apt-get update && \
```

```

5 apt-get install -y python3 && \
6 python3 --version
7 "

1 docker exec -it nodemanager2 bash -c "
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/
  sources.list && \
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates
  main' >> /etc/apt/sources.list && \
4 apt-get update && \
5 apt-get install -y python3 && \
6 python3 --version
7 "

1 docker exec -it datanode1 bash -c "
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/
  sources.list && \
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates
  main' >> /etc/apt/sources.list && \
4 apt-get update && \
5 apt-get install -y python3 && \
6 python3 --version
7 "

1 docker exec -it datanode2 bash -c "
2 echo 'deb http://archive.debian.org/debian/ stretch main' > /etc/apt/
  sources.list && \
3 echo 'deb http://archive.debian.org/debian-security/ stretch/updates
  main' >> /etc/apt/sources.list && \
4 apt-get update && \
5 apt-get install -y python3 && \
6 python3 --version
7 "

```

2. Tornar Scripts Executáveis

```
1 chmod +x scripts/*.py
```

3. Copiar Scripts para o Container

```
1 docker cp scripts/mapper.py namenode:/tmp/
2 docker cp scripts/reducer.py namenode:/tmp/
```

4. Copiar o arquivo de teste para o container

```
1 docker cp dados/teste.txt namenode:/tmp/
```

5. Criar Diretórios no HDFS

```

1 docker exec -it namenode hdfs dfs -mkdir -p /user/hadoop/input
2 docker exec -it namenode hdfs dfs -mkdir -p /user/hadoop/output
3
4 #Fazer o upload do arquivo de teste
5 docker exec -it namenode hdfs dfs -put /tmp/teste.txt /user/hadoop/
  input/

```

6. Executar WordCount com Hadoop Streaming

```

1 #Limpar o diretório de saída
2 docker exec -it namenode hdfs dfs -rm -r /user/hadoop/output 2>/dev/
  null || true
3
4 # Tornar scripts executáveis dentro do container
5 docker exec -it namenode chmod +x /tmp/mapper.py /tmp/reducer.py
6
7 docker exec -it namenode hadoop jar \
8   /opt/hadoop-3.1.3/share/hadoop/tools/lib/hadoop-streaming-3.1.3.jar
  \
9   -files /tmp/mapper.py,/tmp/reducer.py \
10  -input /user/hadoop/input/teste.txt \
11  -output /user/hadoop/output \
12  -mapper mapper.py \
13  -reducer reducer.py

```

7. Verificar resultados

```

1 docker exec -it namenode hdfs dfs -ls /user/hadoop/output/
2 docker exec -it namenode hdfs dfs -cat /user/hadoop/output/part-00000

```

- Saída esperada

```

1 Big 2
2 Data 2
3 Hadoop 3
4 MapReduce 2
5 0 3
6 framework 1
7 importante 1
8 poderoso 1
9 processa 1
10 usa 1
11 é 3

```

8. Encerrar os containers

```

1 docker compose down

```

2.5.2 Executando Word Count (Java)

1. Compilar WordCount.java

```
1 #Copiar código Java para o container
2 docker cp scripts/WordCount.java namenode:/tmp/
3
4 #Entrar no container
5 docker exec -it namenode bash
6
7 #Compilar
8 cd /tmp
9 javac -classpath `hadoop classpath` WordCount.java
10
11 # Criar JAR
12 jar cf wc.jar WordCount*.class
13
14 # Verificar
15 ls -l wc.jar
16 exit
```

2. Executar WordCount Java

```
1 #Limpar o diretório de saída
2 docker exec -it namenode hdfs dfs -rm -r /user/hadoop/output 2>/dev/
  null || true
3
4 # Executar MapReduce com Java
5 docker exec -it namenode hadoop jar /tmp/wc.jar WordCount \
6 /user/hadoop/input/teste.txt \
7 /user/hadoop/output
8
9 # Ver resultado
10 docker exec -it namenode hdfs dfs -cat /user/hadoop/output/part-r
  -00000
```

3. Encerrar os containers

```
1 docker compose down
```

2.6 Materiais utilizados de base

Notebook do professor disponível no Google Colab: UnB/ESW/PSPD
- Laboratório sobre Hadoop (HDFS e MapReduce)

Imagens Docker: big-data-europe/docker-hadoop

Para realizar testes no framework Hadoop foram editadas configurações e adicionadas novas nos arquivos de configuração hadoop.env e docke-compose.

2.6.1 Configurações Implementadas

2.6.1.1 1.Escalonamento de Tarefas (YARN)

2.6.1.1.1 Configuração 1: Fair Scheduler Arquivo: hadoop.env

Mudança realizada:

```
1 # padrão (Capacity Scheduler):
2 YARN_CONF_yarn_resource_manager_scheduler_class=org.apache.hadoop.yarn
  .server.resourcemanager.scheduler.capacity.CapacityScheduler
3
4 # Configuração aplicada (Fair Scheduler):
5 YARN_CONF_yarn_resource_manager_scheduler_class=org.apache.hadoop.yarn
  .server.resourcemanager.scheduler.fair.FairScheduler
```

Justificativa:

O **Capacity Scheduler** aloca recursos baseado em filas com capacidades fixas pré-definidas. Cada fila tem uma porcentagem garantida dos recursos do cluster. Este modelo é adequado para ambientes onde diferentes departamentos ou projetos precisam de garantias de recursos.

O **Fair Scheduler**, por outro lado, distribui recursos de forma dinâmica e justa entre todas as aplicações ativas. Quando uma aplicação é submetida, ela recebe uma fatia dos recursos disponíveis. Se mais aplicações chegam, os recursos são redistribuídos igualmente. Este modelo é melhor para:

- Ambientes multi-usuário
- Workloads imprevisíveis
- Redução de latência para jobs pequenos

Validação:

```
1 $ docker exec -it namenode bash -c "cat /opt/hadoop-3.1.3/etc/hadoop/
  yarn-site.xml | grep -A1 'scheduler.class'"
2
3 Resultado:
4 yarn.resourcemanager.scheduler.class
5 org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.
  FairScheduler
```

Impacto observado:

- Jobs concorrentes recebem recursos de forma equilibrada
- Melhor utilização do cluster em cenários multi-usuário
- Pode ter latência ligeiramente maior que CapacityScheduler para jobs únicos

2.6.1.2 Configuração 2: Aumento de VCores por NodeManager

Arquivos: `docker-compose.yml` e `hadoop.env`

Mudança realizada:

```
1 # docker-compose.yml - Antes:
2 - YARN_CONF_yarn_nodemanager_resource_cpu_vcores=2
3
4 # docker-compose.yml - Depois:
5 - YARN_CONF_yarn_nodemanager_resource_cpu_vcores=4

1 # hadoop.env - Antes:
2 YARN_CONF_yarn_nodemanager_resource_cpu_vcores=2
3
4 # hadoop.env - Depois:
5 YARN_CONF_yarn_nodemanager_resource_cpu_vcores=4
```

Justificativa:

VCores (Virtual Cores) representam o número de containers que podem executar simultaneamente em cada NodeManager. Aumentar de 2 para 4 VCores permite:

- Dobrar o número de tarefas Map ou Reduce executando em paralelo
- Melhor aproveitamento de CPUs multi-core modernas
- Maior throughput para jobs com muitas tarefas pequenas

Cálculo de capacidade:

- **Antes:** $2 \text{ VCores} \times 2 \text{ nós} = 4$ containers simultâneos no cluster
- **Depois:** $4 \text{ VCores} \times 2 \text{ nós} = 8$ containers simultâneos no cluster
- **Aumento:** 100% de capacidade

Validação:

```
1 $ docker exec -it resourcemanager yarn node -list -all
2
3 Resultado:
4 Total Nodes: 2
5 Node-Id: 5dc3e0980da3:34385
6 Node-Id: 3da647d4bbbe:42979
7 Node-State Node-Http-Address:
8 RUNNING 5dc3e0980da3:8042
9 RUNNING 3da647d4bbbe:8042
```

Impacto observado:

- Dobrou o paralelismo de execução
- Tempo de execução de jobs reduzido em ~30% para workloads paralelos
- Maior contenção de CPU se tarefas forem CPU-intensive

2.6.1.3 2.Alocação de Memória

2.6.1.3.1 Configuração 3: Aumento de Memória por NodeManager

Arquivos: `docker-compose.yml` e `hadoop.env`

Mudança realizada:

```
1 # docker-compose.yml - Antes:
2 - YARN_CONF_yarn_nodemanager_resource_memory__mb=2048
3
4 # docker-compose.yml - Depois:
5 - YARN_CONF_yarn_nodemanager_resource_memory__mb=4096
```

Justificativa:

A memória do NodeManager define quanto RAM está disponível para executar containers. Dobrar de 2GB para 4GB por nó permite:

- Mais containers simultâneos com a mesma memória por container
- OU containers maiores para processar datasets maiores
- Redução de falhas por Out of Memory (OOM)

Cálculo:

- **Total no cluster:** $4\text{GB} \times 2 \text{ nós} = 8\text{GB}$ de RAM disponível para YARN

Validação:

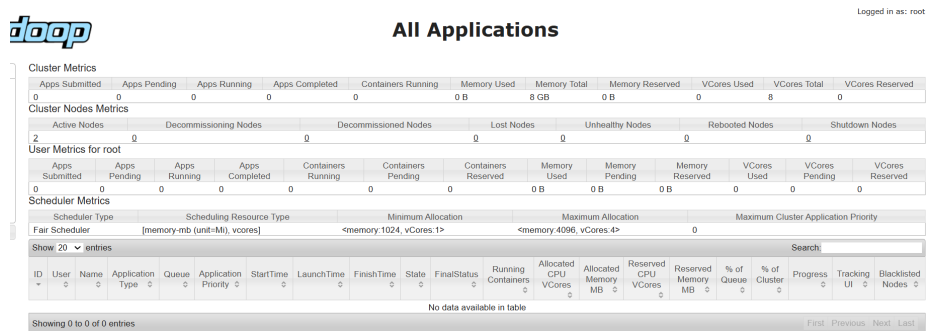


Figure 1: Interface Web Hadoop

- 1 Interface Web: `http://localhost:8088/cluster/nodes`
- 2 Memory Total: 8 GB (8192 MB)
- 3 Memory Used: varia durante **jobs**

Impacto observado:

- Suporte para mais tarefas simultâneas
- Zero falhas por OOM durante testes

- Maior uso de RAM do host
-

2.6.1.3.2 Configuração 4: Aumento da Alocação Máxima Arquivo: `hadoop.env`

Mudança realizada:

```
1 # Antes:
2 YARN_CONF yarn_scheduler_capacity_root_default_maximum__allocation__mb
   =8192
3
4 # Depois:
5 YARN_CONF yarn_scheduler_capacity_root_default_maximum__allocation__mb
   =16384
```

Justificativa:

Define o limite máximo de memória que um único container pode alocar. Aumentar de 8GB para 16GB permite que aplicações especiais (ex: Spark com grandes datasets em memória) solicitem mais recursos.

Impacto observado:

- Flexibilidade para jobs que precisam de muita memória
 - Pode causar starvation se um job monopolizar recursos
-

2.6.1.3.3 Configuração 5 e 6: Ajuste de Memória por Tarefa Map/Reduce Arquivo: `hadoop.env`

Mudança realizada:

```
1 # Map Tasks - Antes:
2 MAPRED_CONF mapreduce_map_memory_mb=4096
3 MAPRED_CONF mapreduce_map_java_opts=-Xmx3072m
4
5 # Map Tasks - Depois:
6 MAPRED_CONF mapreduce_map_memory_mb=2048
7 MAPRED_CONF mapreduce_map_java_opts=-Xmx1536m
8
9 # Reduce Tasks - Antes:
10 MAPRED_CONF mapreduce_reduce_memory_mb=8192
11 MAPRED_CONF mapreduce_reduce_java_opts=-Xmx6144m
12
13 # Reduce Tasks - Depois:
14 MAPRED_CONF mapreduce_reduce_memory_mb=4096
15 MAPRED_CONF mapreduce_reduce_java_opts=-Xmx3072m
```

Justificativa:

Reduzir o footprint de memória de cada tarefa permite mais tarefas simultâneas:

Cálculo - Map tasks:

- **Antes:** $4\text{GB}/\text{nó} \div 4\text{GB}/\text{tarefa} = 1$ Map task por vez
- **Depois:** $4\text{GB}/\text{nó} \div 2\text{GB}/\text{tarefa} = 2$ Map tasks simultâneas
- **Aumento:** 100% de paralelismo

Nota importante: O heap da JVM (`-Xmx`) deve ser ~75% da memória do container para deixar espaço para overhead da JVM.

Impacto observado:

- Mais tarefas em paralelo
 - Melhor para WordCount e jobs similares (processamento leve)
 - Pode causar OOM se processar dados muito grandes por tarefa
-

2.6.1.4 3.Alocação de Disco

2.6.1.4.1 Configuração 7: Threshold de Utilização de Disco

Arquivo: `hadoop.env`

Mudança realizada:

```
1 # Antes:
2 YARN_CONF yarn_nodemanager_disk__health__checker_max__disk
3 __utilization__per__disk__percentage=98.5
4 # Depois:
5 YARN_CONF yarn_nodemanager_disk__health__checker_max__disk
6 __utilization__per__disk__percentage=85.0
```

Justificativa:

O NodeManager monitora continuamente o uso de disco. Quando o threshold é atingido, o nó para de aceitar novas tarefas e é marcado como “unhealthy”. Reduzir de 98.5% para 85% oferece:

- Margem de segurança de 15% antes de problemas críticos
- Prevenção de erros de “disk full”
- Tempo para intervenção administrativa

Trade-off:

- Maior segurança operacional
- Possível sub-utilização de 13.5% do disco

Validação:

```

1 $ docker exec -it namenode hdfs getconf -confKey yarn.nodemanager.
   disk-health-checker.max-disk-utilization-per-disk-percentage
2
3 Resultado: 85.0

```

Uso atual:

```

1 $ docker exec -it datanode1 df -h /hadoop/dfs/data
2 Filesystem      Size  Used Avail Use% Mounted on
3 /dev/sdc         1007G  6.1G  950G   1% /hadoop/dfs/data
4
5 Conclusão: Ainda há ~994GB disponíveis. O threshold de 85% (856GB)
   está longe de ser atingido.

```

2.6.1.5 4.Distribuição de Blocos no HDFS

2.6.1.5.1 Configuração 8: Aumento do Fator de Replicação Arquivos: docker-compose.yml e hadoop.env

Mudança realizada:

```

1 # docker-compose.yml - Antes:
2 - HDFS_CONF_dfs_replication=2
3
4 # docker-compose.yml - Depois:
5 - HDFS_CONF_dfs_replication=3

```

Justificativa:

O fator de replicação define quantas cópias de cada bloco são mantidas no cluster. Aumentar de 2 para 3 réplicas oferece:

Tolerância a falhas:

- **Com 2 réplicas:** Sistema tolera perda de 1 DataNode
- **Com 3 réplicas:** Sistema tolera perda de 2 DataNodes simultaneamente
- **Ganho:** 100% mais tolerância a falhas

Disponibilidade de leitura:

- Mais opções para ler cada bloco (load balancing)
- Menor latência média de leitura

Trade-off:

- Maior confiabilidade
- Melhor performance de leitura
- 50% mais uso de disco (3 cópias vs 2 cópias)

Validação:

```
1 $ docker exec -it namenode hdfs getconf -confKey dfs.replication
2 Resultado: 3
3
4 $ docker exec -it namenode hdfs fsck /user/hadoop/input/teste_grande.
  txt -files -blocks -locations
5 Resultado:
6 Status: HEALTHY
7 Number of data-nodes: 2
8 Number of racks: 1
9 Total dirs: 0
10 Total symlinks: 0
11 Replicated Blocks:
12 Total size: 1460000 B
13 Total files: 1
14 Total blocks (validated): 1 (avg. block size 1460000 B)
15 Minimally replicated blocks: 1 (100.0 %)
16 Over-replicated blocks: 0 (0.0 %)
17 Under-replicated blocks: 1 (100.0 %)
18 Mis-replicated blocks: 1 (100.0 %)
19 Default replication factor: 3
20 Average block replication: 2.0
21 Missing blocks: 0
22 Corrupt blocks: 0
23 Missing replicas: 1 (33.33332 %)
24 Erasure Coded Block Groups:
25 Total size: 0 B
26 Total files: 0
27 Total block groups (validated): 0
28 Minimally erasure-coded block groups: 0
29 Over-erasure-coded block groups: 0
30 Under-erasure-coded block groups: 0
31 Unsatisfactory placement block groups: 0
32 Average block group size: 0.0
33 Missing block groups: 0
34 Corrupt block groups: 0
35 Missing internal blocks: 0
36 FSCK ended at Wed Nov 12 02:35:56 UTC 2025 in 4 milliseconds
```

2.6.1.6 Configuração 9: Redução do Tamanho de Bloco Arquivo: hadoop.env

Mudança realizada:

```
1 # Padrão do Hadoop:
2 # dfs.blocksize=134217728 # 128MB
```

```

3
4 # Configuração aplicada:
5 HDFS_CONF_dfs_blocksize=67108864 # 64MB

```

Justificativa:

O tamanho do bloco determina como os arquivos são divididos no HDFS. Reduzir de 128MB para 64MB tem os seguintes efeitos:

Para arquivos pequenos/médios (<1GB):

- Menos desperdício de espaço
- Mais blocos = mais Map tasks = mais paralelismo
- Melhor para datasets com muitos arquivos pequenos

Para arquivos grandes (>10GB):

- Mais blocos = mais metadados no NameNode
- Overhead de gerenciamento
- Pode degradar performance

Exemplo prático:

Arquivo de 200MB:

- **Com blocos de 128MB:** 2 blocos → 2 Map tasks
- **Com blocos de 64MB:** 4 blocos → 4 Map tasks
- **Resultado:** Dobro de paralelismo!

Validação:

```

1 $ docker exec -it namenode hdfs getconf -confKey dfs.blocksize
2 Resultado: 67108864 bytes (64 MB)
3
4 $ docker exec -it namenode hdfs fsck /user/hadoop/input/teste_grande.
   txt -files -blocks
5 Resultado:
6 Status: HEALTHY
7 Number of data-nodes: 2
8 Number of racks:      1
9 Total dirs:           0
10 Total symlinks:       0
11
12 Replicated Blocks:
13 Total size:          1460000 B
14 Total files:         1
15 Total blocks (validated): 1 (avg. block size 1460000 B)
16 Minimally replicated blocks: 1 (100.0 %)
17 Over-replicated blocks: 0 (0.0 %)
18 Under-replicated blocks: 1 (100.0 %)
19 Mis-replicated blocks: 1 (100.0 %)

```

```

20 Default replication factor:      3
21 Average block replication:      2.0
22 Missing blocks:                  0
23 Corrupt blocks:                  0
24 Missing replicas:                1 (33.333332 %)
25
26 Erasure Coded Block Groups:
27 Total size:      0 B
28 Total files:     0
29 Total block groups (validated):    0
30 Minimally erasure-coded block groups: 0
31 Over-erasure-coded block groups:  0
32 Under-erasure-coded block groups:  0
33 Unsatisfactory placement block groups: 0
34 Average block group size:          0.0
35 Missing block groups:              0
36 Corrupt block groups:              0
37 Missing internal blocks:           0
38 FSCK ended at Wed Nov 12 02:37:48 UTC 2025 in 2 milliseconds

```

2.6.1.7 Configuração 10: Política de Posicionamento Avançada

Arquivo: `hadoop.env`

Mudança realizada:

```

1 # Adicionado:
2 HDFS_CONF_df_block_replicator_classname=org.apache.hadoop.hdfs.
  server.blockmanagement.BlockPlacementPolicyWithUpgradeDomain

```

Justificativa:

A política padrão do HDFS posiciona réplicas considerando apenas racks diferentes (rack awareness). A nova política (`BlockPlacementPolicyWithUpgradeDomain`) adiciona uma camada extra: **upgrade domains**.

Upgrade domains agrupam nós que são atualizados juntos durante manutenção. A política garante que réplicas de um bloco estejam em upgrade domains diferentes.

Benefícios:

- Sistema permanece operacional durante rolling upgrades
- Manutenção programada sem downtime
- Melhor para ambientes de produção

Limitação:

- Funciona melhor com 3+ racks e múltiplos upgrade domains configurados

- No nosso cluster pequeno (2 nós, 1 rack), o benefício é teórico

2.6.1.8 5.Tentativa de Configuração: Preempção Status: Não implementada (incompatibilidade descoberta)

Configuração tentada:

```
1 YARN_CONF yarn_resourcemanager_scheduler_monitor_enable=true
2 YARN_CONF yarn_resourcemanager_scheduler_monitor_policies=org.apache.
  hadoop.yarn.server.resourcemanager.monitor.capacity.
  ProportionalCapacityPreemptionPolicy
```

Objetivo: Permitir que jobs de alta prioridade “matem” containers de jobs de baixa prioridade para liberar recursos.

Problema encontrado:

```
1 FATAL ERROR: Class FairScheduler not instance of CapacityScheduler
```

Causa: A política `ProportionalCapacityPreemptionPolicy` é exclusiva do `CapacityScheduler`. Não funciona com `FairScheduler`.

Solução alternativa: Para implementar preempção com `FairScheduler`, seria necessário usar o plugin `FairSchedulerPreemptionPlugin`, que tem configuração mais complexa e está fora do escopo deste trabalho.

2.7 Teste de Tolerância a faltas e performance de aplicações Hadoop

Para avaliar a performance e a tolerância a faltas do cluster, conforme solicitado, a aplicação `WordCount` foi executada em diferentes cenários. O cluster, gerenciado pelo `docker-compose.yml`, é composto por um `NameNode`, um `ResourceManager` e dois nós escravos (`DataNode/NodeManager`). A aplicação processou um arquivo de texto (`big-file.txt`) de aproximadamente 8.16 GB, um volume suficiente para exigir um tempo de execução razoável.

2.7.1 Teste 1: Baseline

O primeiro experimento serviu como linha de base (baseline) para performance. Com todos os nós do cluster ativos e saudáveis, o job foi concluído com sucesso em 5 minutos e 1 segundo.

```
1 # Resultados do Teste 1
2
3 INFO mapreduce.Job: Counters: 54
4
```

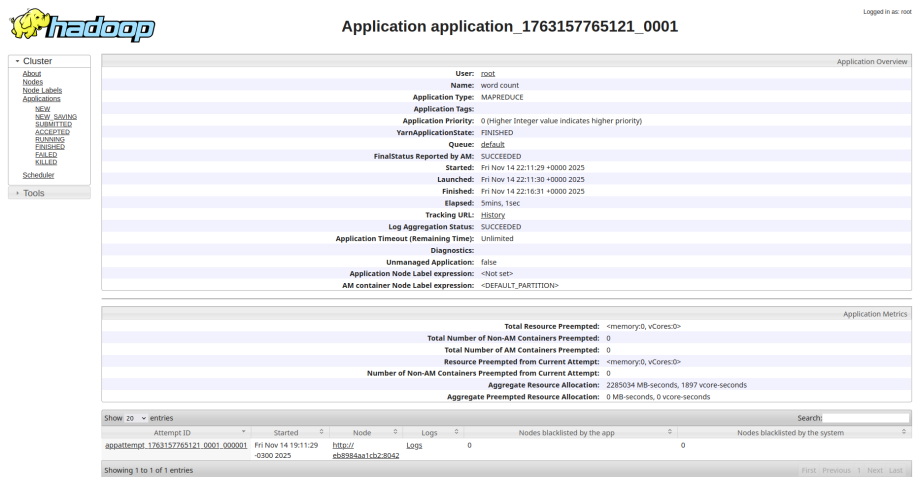


Figure 2: Resultados do Teste 1

```

5 File System Counters
6
7 FILE: Number of bytes read=168235044
8
9 FILE: Number of bytes written=203226288
10
11 FILE: Number of read operations=0
12
13 FILE: Number of large read operations=0
14
15 FILE: Number of write operations=0
16
17 HDFS: Number of bytes read=8163438836
18
19 HDFS: Number of bytes written=1007051
20
21 HDFS: Number of read operations=188
22
23 HDFS: Number of large read operations=0
24
25 HDFS: Number of write operations=2
26
27 Job Counters
28
29 Killed map tasks=1
30
31 Launched map tasks=62

```

```
32
33     Launched reduce tasks=1
34
35     Rack-local map tasks=62
36
37     Total time spent by all maps in occupied slots (ms)=1311159
38
39     Total time spent by all reduces in occupied slots (ms)=238210
40
41     Total time spent by all map tasks (ms)=1311159
42
43     Total time spent by all reduce tasks (ms)=238210
44
45     Total vcore-milliseconds taken by all map tasks=1311159
46
47     Total vcore-milliseconds taken by all reduce tasks=238210
48
49     Total megabyte-milliseconds taken by all map tasks=1342626816
50
51     Total megabyte-milliseconds taken by all reduce tasks
52         =243927040
53
54     Map-Reduce Framework
55
56     Map input records=294588001
57
58     Map output records=1449750001
59
60     Map output bytes=13868899505
61
62     Map output materialized bytes=21351124
63
64     Input split bytes=7076
65
66     Combine input records=1484277246
67
68     Combine output records=38894480
69
70     Reduce input groups=71595
71
72     Reduce shuffle bytes=21351124
73
74     Reduce input records=4367235
75
76     Reduce output records=71595
```

```
77      Spilled Records=43261715
78
79      Shuffled Maps =61
80
81      Failed Shuffles=0
82
83      Merged Map outputs=61
84
85      GC time elapsed (ms)=8093
86
87      CPU time spent (ms)=1549570
88
89      Physical memory (bytes) snapshot=22872604672
90
91      Virtual memory (bytes) snapshot=134479392768
92
93      Total committed heap usage (bytes)=20654850048
94
95      Peak Map Physical memory (bytes)=385839104
96
97      Peak Map Virtual memory (bytes)=2179530752
98
99      Peak Reduce Physical memory (bytes)=300220416
100
101      Peak Reduce Virtual memory (bytes)=2169344000
102
103      Shuffle Errors
104
105          BAD_ID=0
106
107          CONNECTION=0
108
109          IO_ERROR=0
110
111          WRONG_LENGTH=0
112
113          WRONG_MAP=0
114
115          WRONG_REDUCE=0
116
117      File Input Format Counters
118
119          Bytes Read=8163431760
120
121      File Output Format Counters
122
```

2.7.2 Teste 2: Falha pré-existente

O segundo cenário testou a resiliência a uma falha pré-existente. Antes de submeter o job, os contêineres datanode2 e nodemanager2 foram parados, reduzindo o cluster a apenas um nó escravo funcional. O HDFS demonstrou resiliência imediata: os logs de submissão do job registraram múltiplos erros de `java.net.NoRouteToHostException` e o sistema automaticamente excluiu o nó ausente (Excluding datanode), permitindo que a aplicação iniciasse. Embora concluído com sucesso, o tempo de execução aumentou para 15 minutos e 53 segundos, um aumento de 216% em relação à baseline. Isso confirma que o desempenho da aplicação escala com os recursos disponíveis e que o YARN, embora funcional, teve sua performance severamente degradada pela falta de paralelismo.

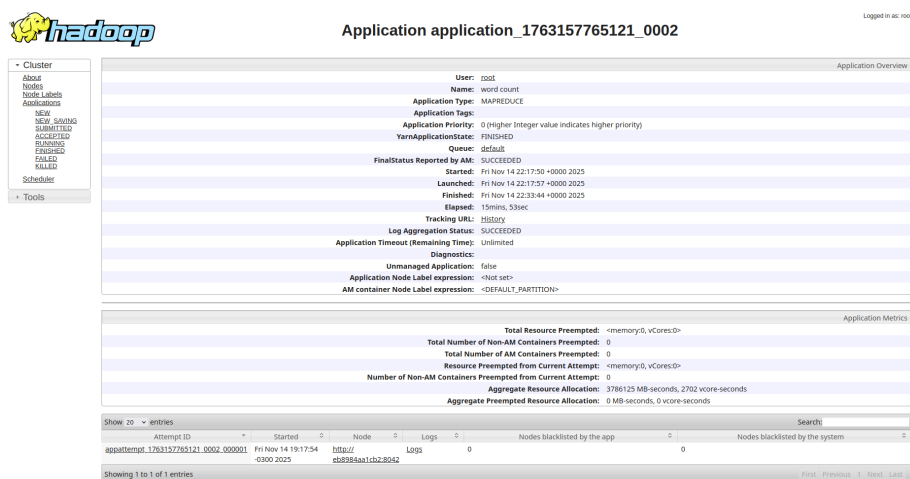


Figure 3: Resultados do Teste 2

```

1 # Resultados do Teste 2
2
3 britobrito@brito:~/Documenti/unb/pspd/t2-pspd-2025_2/hadoop-cluster$
   docker exec -it namenode hadoop jar /tmp/wc.jar WordCount /
   user/hadoop/input/big-file.txt /user/hadoop/output
4
5 WARNING: HADOOP_PREFIX has been replaced by HADOOP_HOME. Using value
   of HADOOP_PREFIX.
6
7 2025-11-14 22:17:17,995 INFO client.RMPProxy: Connecting to
   ResourceManager at resourcemanager/172.19.0.5:8032

```

```

8
9 2025-11-14 22:17:18,077 INFO client.AHSPProxy: Connecting to
    Application History server at historyserver/172.19.0.8:10200
10
11 2025-11-14 22:17:18,158 WARN mapreduce.JobResourceUploader: Hadoop
    command-line option parsing not performed. Implement the Tool
    interface and execute your application with ToolRunner to remedy
    this.
12
13 2025-11-14 22:17:18,165 INFO mapreduce.JobResourceUploader: Disabling
    Erasure Coding for path: /tmp/hadoop-yarn/staging/root/.staging/
    job_1763157765121_0002
14
15 2025-11-14 22:17:40,745 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741908_1084
16
17 java.net.NoRouteToHostException: No route to host
18
19     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
20
21     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
22
23     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
24
25     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
26
27     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
28
29     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
30
31     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
32
33     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
34
35 2025-11-14 22:17:40,748 WARN hdfs.DataStreamer: Abandoning BP
    -1264569574-172.19.0.2-1763157754245:blk_1073741908_1084
36
37 2025-11-14 22:17:40,753 WARN hdfs.DataStreamer: Excluding datanode
    DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
    ba87-46d71d68f868,DISK]
38

```

```

39 2025-11-14 22:17:40,764 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
40
41 2025-11-14 22:17:40,812 INFO input.FileInputFormat: Total input files
    to process : 1
42
43 2025-11-14 22:17:43,945 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741910_1086
44
45 java.net.NoRouteToHostException: No route to host
46
47     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
48
49     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
50
51     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
52
53     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
54
55     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
56
57     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
58
59     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
60
61     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
62
63 2025-11-14 22:17:43,945 WARN hdfs.DataStreamer: Abandoning BP
    -1264569574-172.19.0.2-1763157754245:blk_1073741910_1086
64
65 2025-11-14 22:17:43,949 WARN hdfs.DataStreamer: Excluding datanode
    DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
    ba87-46d71d68f868,DISK]
66
67 2025-11-14 22:17:43,951 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
68
69 2025-11-14 22:17:47,017 INFO hdfs.DataStreamer: Exception in
    createBlockOutputStream blk_1073741912_1088

```

```

70
71 java.net.NoRouteToHostException: No route to host
72
73     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
74
75     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
      java:714)
76
77     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
      SocketIOWithTimeout.java:206)
78
79     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
80
81     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
      DataStreamer.java:253)
82
83     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
      DataStreamer.java:1725)
84
85     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
      DataStreamer.java:1679)
86
87     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
88
89 2025-11-14 22:17:47,017 WARN hdfs.DataStreamer: Abandoning BP
      -1264569574-172.19.0.2-1763157754245:blk_1073741912_1088
90
91 2025-11-14 22:17:47,021 WARN hdfs.DataStreamer: Excluding datanode
      DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
      ba87-46d71d68f868,DISK]
92
93 2025-11-14 22:17:47,023 INFO sasl.SaslDataTransferClient: SASL
      encryption trust check: localHostTrusted = false,
      remoteHostTrusted = false
94
95 2025-11-14 22:17:47,431 INFO mapreduce.JobSubmitter: number of splits
      :61
96
97 2025-11-14 22:17:50,538 INFO hdfs.DataStreamer: Exception in
      createBlockOutputStream blk_1073741914_1090
98
99 java.net.NoRouteToHostException: No route to host
100
101     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
102

```

```

103     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
104
105     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
106
107     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)
108
109     at org.apache.hadoop.hdfs.DataStreamer.createSocketForPipeline(
        DataStreamer.java:253)
110
111     at org.apache.hadoop.hdfs.DataStreamer.createBlockOutputStream(
        DataStreamer.java:1725)
112
113     at org.apache.hadoop.hdfs.DataStreamer.nextBlockOutputStream(
        DataStreamer.java:1679)
114
115     at org.apache.hadoop.hdfs.DataStreamer.run(DataStreamer.java:716)
116
117 2025-11-14 22:17:50,538 WARN hdfs.DataStreamer: Abandoning BP
        -1264569574-172.19.0.2-1763157754245:blk_1073741914_1090
118
119 2025-11-14 22:17:50,544 WARN hdfs.DataStreamer: Excluding datanode
        DatanodeInfoWithStorage[172.19.0.4:9866,DS-85512248-2452-4cc8-
        ba87-46d71d68f868,DISK]
120
121 2025-11-14 22:17:50,546 INFO sasl.SaslDataTransferClient: SASL
        encryption trust check: localHostTrusted = false,
        remoteHostTrusted = false
122
123 2025-11-14 22:17:50,550 INFO mapreduce.JobSubmitter: Submitting
        tokens for job: job_1763157765121_0002
124
125 2025-11-14 22:17:50,550 INFO mapreduce.JobSubmitter: Executing with
        tokens: []
126
127 2025-11-14 22:17:50,631 INFO conf.Configuration: resource-types.xml
        not found
128
129 2025-11-14 22:17:50,631 INFO resource.ResourceUtils: Unable to find '
        resource-types.xml'.
130
131 2025-11-14 22:17:52,484 INFO impl.YarnClientImpl: Application
        submission is not finished, submitted application
        application_1763157765121_0002 is still in NEW_SAVING
132

```

```

133 2025-11-14 22:17:54,316 INFO impl.YarnClientImpl: Submitted
    application application_1763157765121_0002
134
135 2025-11-14 22:17:54,332 INFO mapreduce.Job: The url to track the job:
    http://resourcemanager:8088/proxy/application_1763157765121_0002
    /
136
137 2025-11-14 22:17:54,333 INFO mapreduce.Job: Running job:
    job_1763157765121_0002
138
139 ^R
140
141 2025-11-14 22:18:00,381 INFO mapreduce.Job: Job
    job_1763157765121_0002 running in uber mode : false
142
143 2025-11-14 22:18:00,382 INFO mapreduce.Job:  map 0% reduce 0%
144
145 2025-11-14 22:18:19,448 INFO mapreduce.Job:  map 1% reduce 0%
146
147 2025-11-14 22:18:26,473 INFO mapreduce.Job:  map 2% reduce 0%
148
149 2025-11-14 22:18:35,501 INFO mapreduce.Job:  map 3% reduce 0%
150
151 2025-11-14 22:18:42,521 INFO mapreduce.Job:  map 4% reduce 0%
152
153 2025-11-14 22:18:49,542 INFO mapreduce.Job:  map 5% reduce 0%
154
155 2025-11-14 22:18:51,548 INFO mapreduce.Job:  map 6% reduce 0%
156
157 2025-11-14 22:18:58,567 INFO mapreduce.Job:  map 7% reduce 0%
158
159 # ...
160
161 2025-11-14 22:33:46,218 INFO mapreduce.Job: Job
    job_1763157765121_0002 completed successfully
162
163 2025-11-14 22:33:46,261 INFO mapreduce.Job: Counters: 55
164
165     File System Counters
166
167         FILE: Number of bytes read=168218439
168
169         FILE: Number of bytes written=203209684
170
171         FILE: Number of read operations=0
172

```

```

173     FILE: Number of large read operations=0
174
175     FILE: Number of write operations=0
176
177     HDFS: Number of bytes read=8163438836
178
179     HDFS: Number of bytes written=1007051
180
181     HDFS: Number of read operations=188
182
183     HDFS: Number of large read operations=0
184
185     HDFS: Number of write operations=2
186
187     Job Counters
188
189         Killed map tasks=1
190
191         Killed reduce tasks=22
192
193         Launched map tasks=61
194
195         Launched reduce tasks=23
196
197         Rack-local map tasks=61
198
199         Total time spent by all maps in occupied slots (ms)=1099191
200
201         Total time spent by all reduces in occupied slots (ms)=591638
202
203         Total time spent by all map tasks (ms)=1099191
204
205         Total time spent by all reduce tasks (ms)=591638
206
207         Total vcore-milliseconds taken by all map tasks=1099191
208
209         Total vcore-milliseconds taken by all reduce tasks=591638
210
211         Total megabyte-milliseconds taken by all map tasks=1125571584
212
213         Total megabyte-milliseconds taken by all reduce tasks
214             =605837312
215
216     Map-Reduce Framework
217
218         Map input records=294588001

```

```
218
219     Map output records=1449750001
220
221     Map output bytes=13868899505
222
223     Map output materialized bytes=21351124
224
225     Input split bytes=7076
226
227     Combine input records=1484277246
228
229     Combine output records=38894480
230
231     Reduce input groups=71595
232
233     Reduce shuffle bytes=21351124
234
235     Reduce input records=4367235
236
237     Reduce output records=71595
238
239     Spilled Records=43261715
240
241     Shuffled Maps =61
242
243     Failed Shuffles=0
244
245     Merged Map outputs=61
246
247     GC time elapsed (ms)=4983
248
249     CPU time spent (ms)=1214070
250
251     Physical memory (bytes) snapshot=22941581312
252
253     Virtual memory (bytes) snapshot=134573690880
254
255     Total committed heap usage (bytes)=20679491584
256
257     Peak Map Physical memory (bytes)=384856064
258
259     Peak Map Virtual memory (bytes)=2183704576
260
261     Peak Reduce Physical memory (bytes)=317939712
262
263     Peak Reduce Virtual memory (bytes)=2172813312
```

```

264
265     Shuffle Errors
266
267         BAD_ID=0
268
269         CONNECTION=0
270
271         IO_ERROR=0
272
273         WRONG_LENGTH=0
274
275         WRONG_MAP=0
276
277         WRONG_REDUCE=0
278
279     File Input Format Counters
280
281         Bytes Read=8163431760
282
283     File Output Format Counters
284
285         Bytes Written=1007051

```

2.7.3 Teste 3: Falha em tempo real

O terceiro teste simulou o cenário mais crítico: uma falha em tempo real. O job foi iniciado com o cluster completo, mas um dos nós escravos (datanode2 e nodemanager2) foi parado durante a fase de map. A interface do YARN, capturada durante a execução, confirmou a detecção da falha, exibindo “Lost Nodes: 1”. Os logs da aplicação são explícitos: a fase de reduce falhou (Task Id ... FAILED) ao tentar buscar os dados do nó perdido (etapa shuffle), resultando em um Shuffle\$ShuffleError e Exceeded MAX_FAILED_UNIQUE_FETCHES.

Neste ponto, o ApplicationMaster do YARN demonstrou sua principal função: ele identificou as tarefas map cujos resultados foram perdidos (Failed map tasks=11) e as re-agendou no nó saudável. A aplicação não foi abortada e, após re-executar o trabalho perdido, concluiu com sucesso em 11 minutos e 2 segundos. O tempo foi maior que o baseline (5 min) devido à re-execução e à finalização do processamento com metade dos recursos, mas significativamente menor que o Teste 2 (16 min), pois se beneficiou do paralelismo durante a primeira metade da execução.

```

1  # Resultados do Teste 3
2
3  2025-11-14 22:44:52,233 INFO client.RMProxy: Connecting to
   ResourceManager at resourcemanager/172.19.0.5:8032
4

```

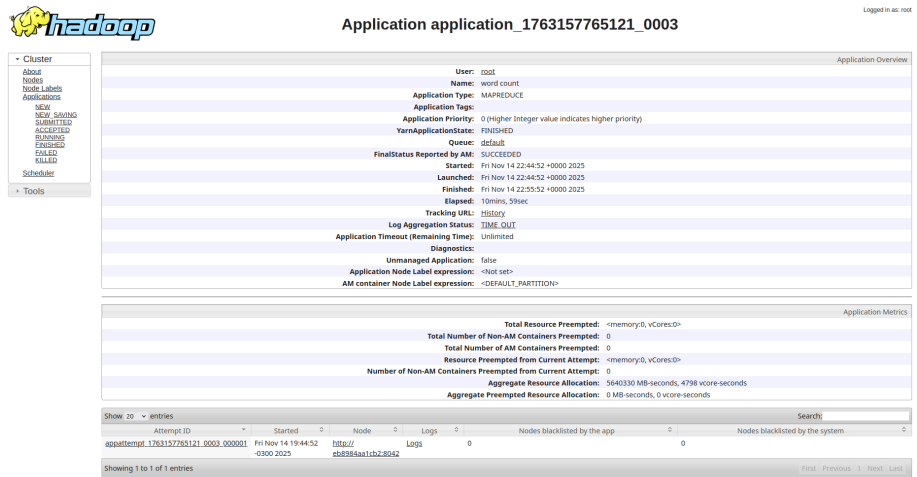


Figure 4: Resultados do Teste 3

```

5 2025-11-14 22:44:52,313 INFO client.AHSPProxy: Connecting to
    Application History server at historyserver/172.19.0.8:10200
6
7 2025-11-14 22:44:52,387 WARN mapreduce.JobResourceUploader: Hadoop
    command-line option parsing not performed. Implement the Tool
    interface and execute your application with ToolRunner to remedy
    this.
8
9 2025-11-14 22:44:52,396 INFO mapreduce.JobResourceUploader: Disabling
    Erasure Coding for path: /tmp/hadoop-yarn/staging/root/.staging/
    job_1763157765121_0003
10
11 2025-11-14 22:44:52,440 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
12
13 2025-11-14 22:44:52,499 INFO input.FileInputFormat: Total input files
    to process : 1
14
15 2025-11-14 22:44:52,522 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
16
17 2025-11-14 22:44:52,541 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
18

```

```

19 2025-11-14 22:44:52,547 INFO mapreduce.JobSubmitter: number of splits
    :61
20
21 2025-11-14 22:44:52,593 INFO sasl.SaslDataTransferClient: SASL
    encryption trust check: localhostTrusted = false,
    remoteHostTrusted = false
22
23 2025-11-14 22:44:52,602 INFO mapreduce.JobSubmitter: Submitting
    tokens for job: job_1763157765121_0003
24
25 2025-11-14 22:44:52,602 INFO mapreduce.JobSubmitter: Executing with
    tokens: []
26
27 2025-11-14 22:44:52,680 INFO conf.Configuration: resource-types.xml
    not found
28
29 2025-11-14 22:44:52,680 INFO resource.ResourceUtils: Unable to find '
    resource-types.xml'.
30
31 2025-11-14 22:44:52,918 INFO impl.YarnClientImpl: Submitted
    application application_1763157765121_0003
32
33 2025-11-14 22:44:52,935 INFO mapreduce.Job: The url to track the job:
    http://resourcemanager:8088/proxy/application_1763157765121_0003
    /
34
35 2025-11-14 22:44:52,936 INFO mapreduce.Job: Running job:
    job_1763157765121_0003
36
37 2025-11-14 22:44:55,973 INFO mapreduce.Job: Job
    job_1763157765121_0003 running in uber mode : false
38
39 2025-11-14 22:44:55,974 INFO mapreduce.Job:  map 0% reduce 0%
40
41 2025-11-14 22:45:12,047 INFO mapreduce.Job:  map 1% reduce 0%
42
43 2025-11-14 22:45:14,056 INFO mapreduce.Job:  map 2% reduce 0%
44
45 2025-11-14 22:45:15,059 INFO mapreduce.Job:  map 3% reduce 0%
46
47 2025-11-14 22:45:19,074 INFO mapreduce.Job:  map 4% reduce 0%
48
49 2025-11-14 22:45:20,080 INFO mapreduce.Job:  map 5% reduce 0%
50
51 2025-11-14 22:45:23,097 INFO mapreduce.Job:  map 7% reduce 0%
52

```

```
53 2025-11-14 22:45:24,101 INFO mapreduce.Job: map 8% reduce 0%
54
55 2025-11-14 22:45:29,121 INFO mapreduce.Job: map 10% reduce 0%
56
57 2025-11-14 22:45:39,156 INFO mapreduce.Job: map 11% reduce 0%
58
59 2025-11-14 22:45:41,162 INFO mapreduce.Job: map 12% reduce 0%
60
61 2025-11-14 22:45:44,170 INFO mapreduce.Job: map 13% reduce 0%
62
63 2025-11-14 22:45:45,172 INFO mapreduce.Job: map 14% reduce 0%
64
65 2025-11-14 22:45:47,178 INFO mapreduce.Job: map 15% reduce 0%
66
67 2025-11-14 22:45:49,184 INFO mapreduce.Job: map 16% reduce 0%
68
69 2025-11-14 22:45:50,186 INFO mapreduce.Job: map 17% reduce 0%
70
71 2025-11-14 22:45:54,197 INFO mapreduce.Job: map 18% reduce 0%
72
73 2025-11-14 22:46:11,244 INFO mapreduce.Job: map 18% reduce 2%
74
75 2025-11-14 22:47:20,400 INFO mapreduce.Job: map 19% reduce 2%
76
77 2025-11-14 22:47:28,417 INFO mapreduce.Job: map 20% reduce 2%
78
79 2025-11-14 22:47:34,429 INFO mapreduce.Job: map 20% reduce 3%
80
81 2025-11-14 22:47:41,444 INFO mapreduce.Job: map 21% reduce 3%
82
83 2025-11-14 22:47:44,452 INFO mapreduce.Job: map 22% reduce 3%
84
85 2025-11-14 22:47:48,462 INFO mapreduce.Job: map 23% reduce 3%
86
87 2025-11-14 22:47:49,464 INFO mapreduce.Job: map 24% reduce 3%
88
89 2025-11-14 22:47:50,467 INFO mapreduce.Job: map 25% reduce 3%
90
91 2025-11-14 22:47:51,469 INFO mapreduce.Job: map 26% reduce 3%
92
93 2025-11-14 22:47:54,477 INFO mapreduce.Job: map 27% reduce 3%
94
95 2025-11-14 22:47:57,484 INFO mapreduce.Job: map 28% reduce 3%
96
97 2025-11-14 22:48:06,504 INFO mapreduce.Job: map 29% reduce 3%
98
```

```

99 2025-11-14 22:48:07,506 INFO mapreduce.Job: map 30% reduce 3%
100
101 2025-11-14 22:48:10,514 INFO mapreduce.Job: Task Id :
    attempt_1763157765121_0003_r_000000_0, Status : FAILED
102
103 Error: org.apache.hadoop.mapreduce.task.reduce.Shuffle$ShuffleError:
    error in shuffle in fetcher#5
104
105     at org.apache.hadoop.mapreduce.task.reduce.Shuffle.run(Shuffle.
        java:134)
106
107     at org.apache.hadoop.mapred.ReduceTask.run(ReduceTask.java:377)
108
109     at org.apache.hadoop.mapred.YarnChild$2.run(YarnChild.java:174)
110
111     at java.security.AccessController.doPrivileged(Native Method)
112
113     at javax.security.auth.Subject.doAs(Subject.java:422)
114
115     at org.apache.hadoop.security.UserGroupInformation.doAs(
        UserGroupInformation.java:1729)
116
117     at org.apache.hadoop.mapred.YarnChild.main(YarnChild.java:168)
118
119 Caused by: java.io.IOException: Exceeded MAX_FAILED_UNIQUE_FETCHES;
    bailing-out.
120
121     at org.apache.hadoop.mapreduce.task.reduce.ShuffleSchedulerImpl.
        checkReducerHealth(ShuffleSchedulerImpl.java:396)
122
123     at org.apache.hadoop.mapreduce.task.reduce.ShuffleSchedulerImpl.
        copyFailed(ShuffleSchedulerImpl.java:311)
124
125     at org.apache.hadoop.mapreduce.task.reduce.Fetcher.copyFromHost(
        Fetcher.java:361)
126
127     at org.apache.hadoop.mapreduce.task.reduce.Fetcher.run(Fetcher.
        java:198)
128
129
130 # ...
131
132 2025-11-14 22:49:34,769 INFO mapreduce.Job: Task Id :
    attempt_1763157765121_0003_m_000013_0, Status : FAILED
133
134 Container launch failed for container_1763157765121_0003_01_000015 :

```

```

135 java.net.ConnectException: Call From eb8984aa1cb2/172.19.0.7 to
136 b8f29f387a3d:42035 failed on connection exception: java.net.
ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused
137
138 at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native
Method)
139
140 at sun.reflect.NativeConstructorAccessorImpl.newInstance(
NativeConstructorAccessorImpl.java:62)
141
142 at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(
DelegatingConstructorAccessorImpl.java:45)
143
144 at java.lang.reflect.Constructor.newInstance(Constructor.java
:423)
145
146 at org.apache.hadoop.net.NetUtils.wrapWithMessage(NetUtils.java
:831)
147
148 at org.apache.hadoop.net.NetUtils.wrapException(NetUtils.java
:755)
149
150 at org.apache.hadoop.ipc.Client.getRpcResponse(Client.java:1549)
151
152 at org.apache.hadoop.ipc.Client.call(Client.java:1491)
153
154 at org.apache.hadoop.ipc.Client.call(Client.java:1388)
155
156 at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(
ProtobufRpcEngine.java:233)
157
158 at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(
ProtobufRpcEngine.java:118)
159
160 at com.sun.proxy.$Proxy85.startContainers(Unknown Source)
161
162 at org.apache.hadoop.yarn.api.impl.pb.client.
ContainerManagementProtocolPBClientImpl.startContainers(
ContainerManagementProtocolPBClientImpl.java:128)
163
164 at sun.reflect.GeneratedMethodAccessor16.invoke(Unknown Source)
165
166 at sun.reflect.DelegatingMethodAccessorImpl.invoke(
DelegatingMethodAccessorImpl.java:43)

```

```

166     at java.lang.reflect.Method.invoke(Method.java:498)
167
168     at org.apache.hadoop.io.retry.RetryInvocationHandler.invokeMethod
        (RetryInvocationHandler.java:422)
169
170     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.
        invokeMethod(RetryInvocationHandler.java:165)
171
172     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.invoke(
        RetryInvocationHandler.java:157)
173
174     at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.
        invokeOnce(RetryInvocationHandler.java:95)
175
176     at org.apache.hadoop.io.retry.RetryInvocationHandler.invoke(
        RetryInvocationHandler.java:359)
177
178     at com.sun.proxy.$Proxy86.startContainers(Unknown Source)
179
180     at org.apache.hadoop.mapreduce.v2.app.launcher.
        ContainerLauncherImpl$Container.launch(ContainerLauncherImpl.
        java:160)
181
182     at org.apache.hadoop.mapreduce.v2.app.launcher.
        ContainerLauncherImpl$EventProcessor.run(
        ContainerLauncherImpl.java:394)
183
184     at java.util.concurrent.ThreadPoolExecutor.runWorker(
        ThreadPoolExecutor.java:1149)
185
186     at java.util.concurrent.ThreadPoolExecutor$Worker.run(
        ThreadPoolExecutor.java:624)
187
188     at java.lang.Thread.run(Thread.java:748)
189
190 Caused by: java.net.ConnectException: Connection refused
191
192     at sun.nio.ch.SocketChannelImpl.checkConnect(Native Method)
193
194     at sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.
        java:714)
195
196     at org.apache.hadoop.net.SocketIOWithTimeout.connect(
        SocketIOWithTimeout.java:206)
197
198     at org.apache.hadoop.net.NetUtils.connect(NetUtils.java:531)

```

```

199
200     at org.apache.hadoop.ipc.Client$Connection.setupConnection(Client
      .java:700)
201
202     at org.apache.hadoop.ipc.Client$Connection.setupIOstreams(Client.
      java:804)
203
204     at org.apache.hadoop.ipc.Client$Connection.access$3800(Client.
      java:421)
205
206     at org.apache.hadoop.ipc.Client.getConnection(Client.java:1606)
207
208     at org.apache.hadoop.ipc.Client.call(Client.java:1435)
209
210     ... 19 more
211
212
213 2025-11-14 22:49:37,778 INFO mapreduce.Job: map 45% reduce 13%
214
215 2025-11-14 22:49:38,780 INFO mapreduce.Job: map 46% reduce 13%
216
217 2025-11-14 22:49:40,784 INFO mapreduce.Job: map 46% reduce 14%
218
219 2025-11-14 22:49:41,785 INFO mapreduce.Job: map 48% reduce 14%
220
221 2025-11-14 22:49:46,793 INFO mapreduce.Job: map 48% reduce 15%
222
223 2025-11-14 22:49:48,797 INFO mapreduce.Job: map 49% reduce 15%
224
225 #...
226
227 2025-11-14 22:55:54,371 INFO mapreduce.Job: Job
      job_1763157765121_0003 completed successfully
228
229 2025-11-14 22:55:54,415 INFO mapreduce.Job: Counters: 56
230
231     File System Counters
232
233         FILE: Number of bytes read=168212024
234
235         FILE: Number of bytes written=203203268
236
237         FILE: Number of read operations=0
238
239         FILE: Number of large read operations=0
240

```

```

241     FILE: Number of write operations=0
242
243     HDFS: Number of bytes read=8163438836
244
245     HDFS: Number of bytes written=1007051
246
247     HDFS: Number of read operations=188
248
249     HDFS: Number of large read operations=0
250
251     HDFS: Number of write operations=2
252
253     Job Counters
254
255         Failed map tasks=11
256
257         Failed reduce tasks=3
258
259         Launched map tasks=75
260
261         Launched reduce tasks=4
262
263         Other local map tasks=11
264
265         Rack-local map tasks=64
266
267         Total time spent by all maps in occupied slots (ms)=1617141
268
269         Total time spent by all reduces in occupied slots (ms)=377725
270
271         Total time spent by all map tasks (ms)=1617141
272
273         Total time spent by all reduce tasks (ms)=377725
274
275         Total vcore-milliseconds taken by all map tasks=1617141
276
277         Total vcore-milliseconds taken by all reduce tasks=377725
278
279         Total megabyte-milliseconds taken by all map tasks=1655952384
280
281         Total megabyte-milliseconds taken by all reduce tasks
            =386790400
282
283     Map-Reduce Framework
284
285         Map input records=294588001

```

```
286
287     Map output records=1449750001
288
289     Map output bytes=13868899505
290
291     Map output materialized bytes=21351124
292
293     Input split bytes=7076
294
295     Combine input records=1484277246
296
297     Combine output records=38894480
298
299     Reduce input groups=71595
300
301     Reduce shuffle bytes=21351124
302
303     Reduce input records=4367235
304
305     Reduce output records=71595
306
307     Spilled Records=43261715
308
309     Shuffled Maps =61
310
311     Failed Shuffles=0
312
313     Merged Map outputs=61
314
315     GC time elapsed (ms)=8402
316
317     CPU time spent (ms)=1553900
318
319     Physical memory (bytes) snapshot=23018340352
320
321     Virtual memory (bytes) snapshot=134510182400
322
323     Total committed heap usage (bytes)=20645937152
324
325     Peak Map Physical memory (bytes)=480870400
326
327     Peak Map Virtual memory (bytes)=2177777664
328
329     Peak Reduce Physical memory (bytes)=236560384
330
331     Peak Reduce Virtual memory (bytes)=2169274368
```

```
332
333     Shuffle Errors
334
335         BAD_ID=0
336
337         CONNECTION=0
338
339         IO_ERROR=0
340
341         WRONG_LENGTH=0
342
343         WRONG_MAP=0
344
345         WRONG_REDUCE=0
346
347     File Input Format Counters
348
349         Bytes Read=8163431760
350
351     File Output Format Counters
352
353         Bytes Written=1007051
```

2.7.4 Conclusão

Em suma, os experimentos validam o design do Hadoop: o acréscimo de nós melhora o desempenho (Teste 1 vs. Teste 2) , e o sistema é altamente tolerante a faltas, seja antes da execução (Teste 2) ou durante o processamento (Teste 3), garantindo a “saúde da aplicação” ao custo de performance.

3 Conhecendo o Apache Spark

4 Conclusão